

课堂行为理解系统 - 产品设计文档

文档版本: 1.0

编写日期: 2026年4月20日

系统名称: Miloco + OpenClaw 课堂行为理解系统

工作流程: 摄像头采集 → 视频处理 → 行为识别 → 数据统计 → 报告生成

目录

- 系统概述
- 系统架构
- 核心模块设计
- 数据设计
- API设计
- 大模型选择
- 部署方案
- 错误处理与恢复
- 日志与调试
- 实施计划

系统概述

1.1 目标

构建一个基于视觉大模型的课堂行为理解系统，通过以下流程实现课堂行为的自动识别与分析：

米家摄像头 → Miloco(RTSP) → OpenClaw(采样) → 视觉模型(识别) → 统计分析 → 文本报告

1.2 核心价值

维度	说明
自动化	无需手工统计课堂行为
高效性	秒级得到课堂行为分析结果
可复用	支持历史对比分析
易扩展	新行为识别可平滑扩展

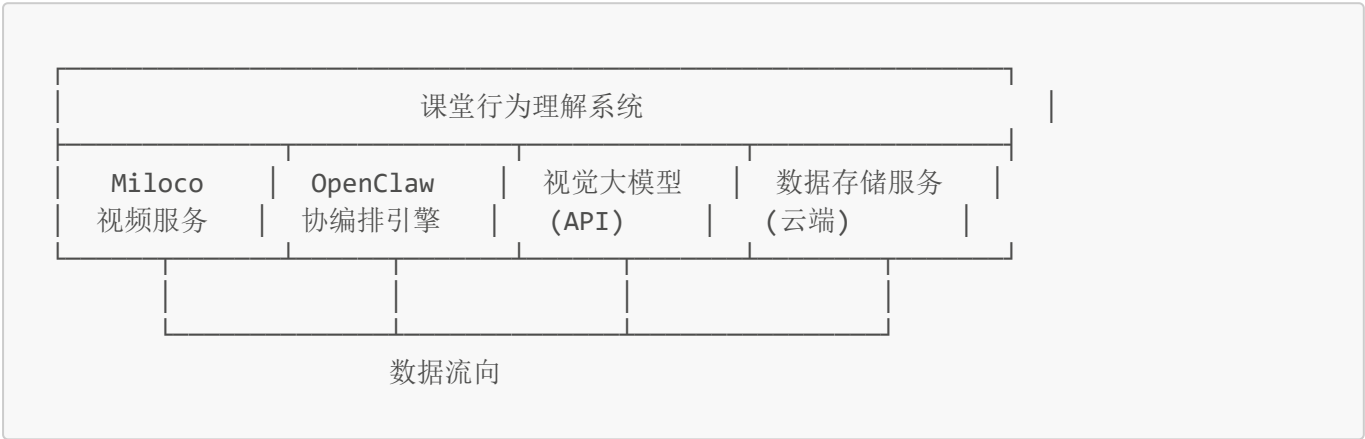
1.3 核心指标

指标	定义	公式
专注率	学生保持专注的比例	记笔记人数 / 总人数

指标	定义	公式
分心率	学生分散注意力的比例	$(\text{睡觉} +) / \text{总人数}$
活跃度	学生参与互动的比例	$\text{举手人数} / \text{总人数}$

系统架构

2.1 整体架构图

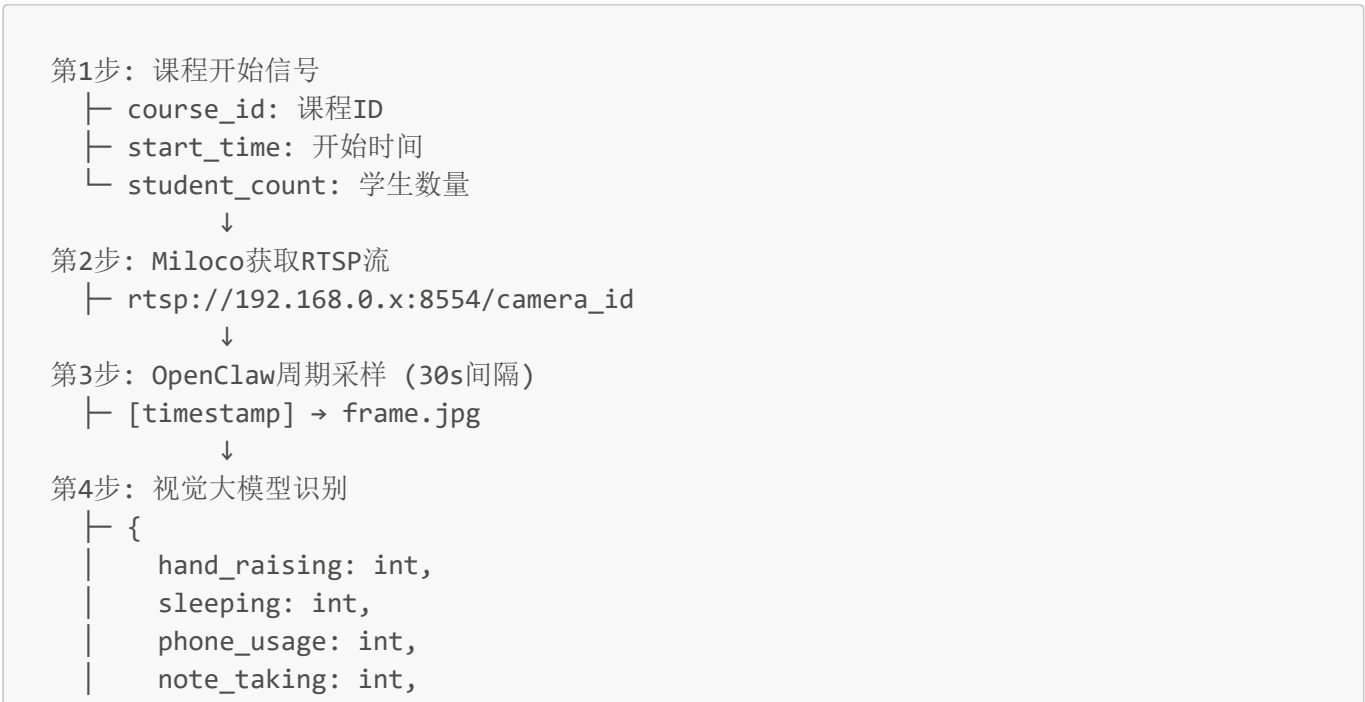


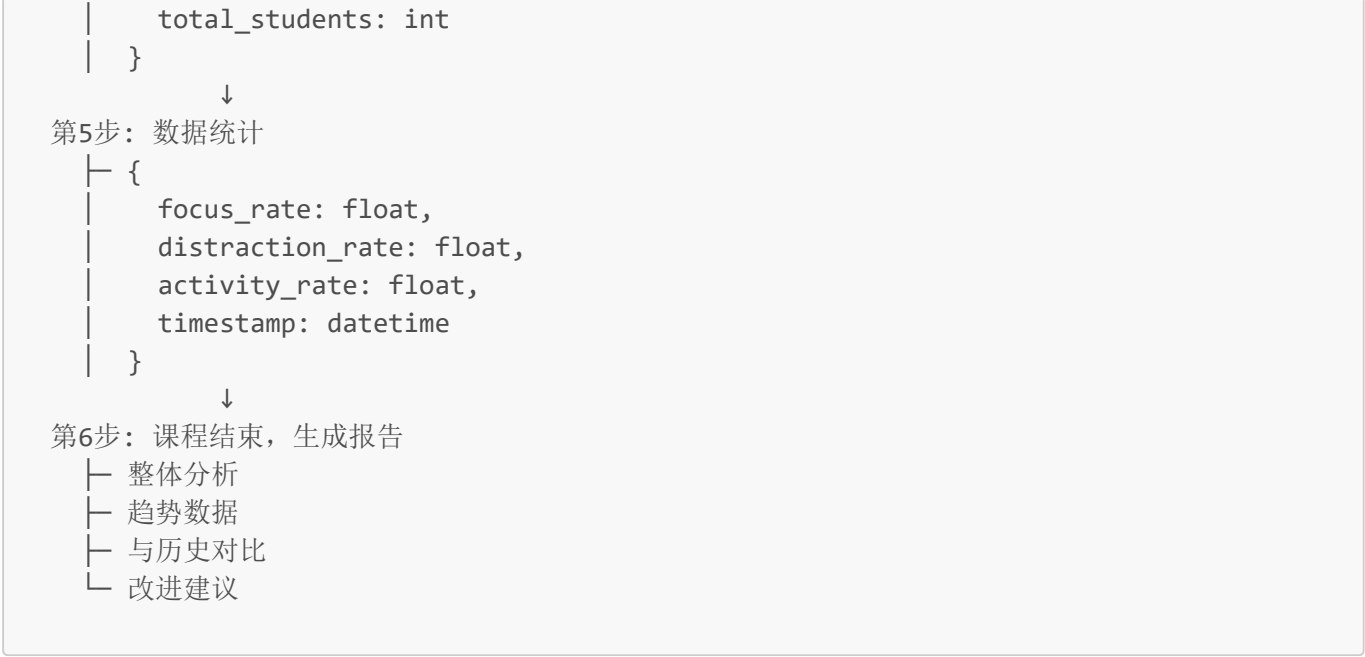
2.2 核心模块

系统分为5个核心模块：

- Miloco视频流服务模块** - 视频采集与RTSP流生成
- OpenClaw采样处理模块** - 视频流解析与帧采样
- 行为识别模块** - 调用视觉大模型识别课堂行为
- 数据统计模块** - 计算课堂状态指标
- 报告生成模块** - 生成课堂分析报告与历史对比

2.3 数据流





核心模块设计

3.1 Miloco视频流服务模块

职责

- 与米家摄像头连接
- 生成标准RTSP视频流
- 提供视频流地址

输入输出

输入：摄像头ID、IP地址、认证信息

输出：rtsp://[server_ip]:[port]/[camera_id]

例：rtsp://192.168.0.136:8554/1027116728

配置参数

参数	说明	示例值
camera_id	摄像头设备ID	1027116728
server_ip	Miloco服务IP	192.168.0.136
rtsp_port	RTSP服务端口	8554
stream_codec	视频编码	H.264

3.2 OpenClaw采样处理模块

职责

- 连接RTSP视频流
- 自动重连管理
- 固定间隔采样
- 帧提取与缓存

采样策略

采样间隔: 30秒

采样方式: 均匀间隔时间采样

单帧大小: 预计500KB-1MB（根据分辨率）

课程时长: 约40-50分钟

计算示例:

课程时长: 45分钟 = 2700秒
采样间隔: 30秒
采样次数: $2700 / 30 = 90$ 帧
总数据量: $90 \text{ 帧} \times 0.8 \text{ MB} = 72 \text{ MB}$ （单节课）

重连机制

连接尝试:

1. 初次连接失败 → 等待2秒后重试
2. 第N次重试失败 → 等待 $\min(2^N, 60)$ 秒
3. 连续失败5次 → 报错, 等待人工处理

连接成功恢复 → 继续正常流程

采样流程

```
# 伪代码
while course_running:
    if current_time - last_sample_time >= 30s:
        frame = capture_frame_from_rtsp()
        if frame:
            timestamp = current_time
            image_path = save_frame(frame, timestamp)
            send_to_vision_model(image_path, timestamp)
            last_sample_time = current_time
        else:
            handle_frame_capture_error()
    sleep(1s)
```

3.3 行为识别模块

职责

- 调用视觉大模型API
- 传递采样帧与识别指令
- 解析模型返回结果
- 结果缓存与验证

识别行为

系统识别4个基本行为：

行为	说明	识别特征
hand_raising	举手	手臂抬起在肩膀以上
sleeping	睡觉	头部低下，闭眼或靠着
phone_usage	玩手机	手持手机在眼前
note_taking	记笔记	俯身、手持笔、低头看文本

视觉模型调用

```
请求格式：
{
  "image_url": "base64_image_or_url",
  "prompt": "请分析当前课堂画面中学生行为，统计以下指标：\n1. 举手人数\n2. 睡觉人数\n3. 玩手机人数\n4. 记笔记人数\n请以JSON格式返回结果。",
  "max_tokens": 500,
  "temperature": 0.3
}

返回格式：
{
  "hand_raising": 2,
  "sleeping": 1,
  "phone_usage": 3,
  "note_taking": 20,
  "total_students": 26,
  "confidence": 0.85,
  "analysis_timestamp": "2026-04-20T14:30:00Z"
}
```

失败处理

```
视觉模型调用失败：
1. 第1次失败 → 等待2秒重试
2. 第2次失败 → 等待5秒重试
```

3. 第3次失败 → 记录错误，跳过本帧，继续处理下一帧
4. 连续10帧失败 → 报错并暂停处理

3.4 数据统计模块

职责

- 汇总多帧识别结果
- 计算课堂状态指标
- 生成统计数据

统计计算

```
def calculate_metrics(frames_data: List[FrameData]) -> Statistics:
    """
    输入：课程内所有采样帧的识别结果
    输出：整个课程的统计指标
    """

    valid_frames = [f for f in frames_data if f.is_valid]

    if not valid_frames:
        raise DataError("无有效数据")

    # 计算平均值
    avg_hand_raising = sum(f.hand_raising for f in valid_frames) /
len(valid_frames)
    avg_sleeping = sum(f.sleeping for f in valid_frames) / len(valid_frames)
    avg_phone_usage = sum(f.phone_usage for f in valid_frames) / len(valid_frames)
    avg_note_taking = sum(f.note_taking for f in valid_frames) / len(valid_frames)

    total_students = valid_frames[0].total_students

    # 计算指标
    focus_rate = (avg_note_taking / total_students) * 100
    distraction_rate = ((avg_sleeping + avg_phone_usage) / total_students) * 100
    activity_rate = (avg_hand_raising / total_students) * 100

    return Statistics(
        focus_rate=focus_rate,
        distraction_rate=distraction_rate,
        activity_rate=activity_rate,
        avg_hand_raising=avg_hand_raising,
        avg_sleeping=avg_sleeping,
        avg_phone_usage=avg_phone_usage,
        avg_note_taking=avg_note_taking,
        total_samples=len(valid_frames),
        invalid_samples=len(frames_data) - len(valid_frames)
    )
```

统计结果示例

```
{
  "course_id": "MATH_10A_20260420",
  "course_name": "高等数学",
  "class_size": 26,
  "total_samples": 90,
  "invalid_samples": 2,
  "sampling_interval_sec": 30,
  "duration_min": 45,
  "statistics": {
    "focus_rate": 76.9,
    "distraction_rate": 15.4,
```

```
"activity_rate": 7.7,  
"avg_hand_raising": 2.0,  
"avg_sleeping": 0.4,  
"avg_phone_usage": 0.8,  
"avg_note_taking": 20.0  
},  
"generated_at": "2026-04-20T15:30:00Z"  
}
```

3.5 报告生成模块

职责

- 基于统计数据生成文本报告
- 提供历史数据对比
- 生成改进建议

报告结构

课堂行为分析报告
【课程信息】 课程：高等数学（MATH_10A） 日期：2026年4月20日 时长：45分钟 人数：26人
【本节课指标】 - 专注率：76.9%（20人） - 分心率：15.4%（4人） - 活跃度：7.7%（2人）
【与历史对比】 - 专注率：↑ 上升 3.2%（前次：73.7%） - 分心率：↓ 下降 2.1%（前次：17.5%） - 活跃度：→ 持平（前次：7.7%）
【行为分析】 本节课整体专注率较高，大部分学生保持记笔记状态，相比前次课程有明显改善。 仍有少量学生存在玩手机现象，需要关注。 课堂互动活跃度偏低，可考虑增加提问机制。
【改进建议】 1. 增加课堂互动环节，提高学生参与度 2. 对分心学生进行个别指导 3. 继续保持现有教学方法效果

报告生成逻辑

```
def generate_report(current_stats: Statistics,
                    previous_stats: Statistics = None) -> str:
    """
    生成课堂分析报告

    参数:
        current_stats: 当前课程统计数据
        previous_stats: 前一课程统计数据（可选）

    返回:
        格式化的文本报告
    """

    report = []

    # 1. 报告头
    report.append(generate_header(current_stats))
```

```
# 2. 本节课指标
report.append(generate_current_metrics(current_stats))

# 3. 历史对比
if previous_stats:
    report.append(generate_comparison(current_stats, previous_stats))

# 4. 行为分析
report.append(generate_analysis(current_stats))

# 5. 改进建议
report.append(generate_suggestions(current_stats))

return "\n".join(report)
```

数据设计

4.1 核心数据模型

4.1.1 采样帧数据 (FrameData)

```
{
  "frame_id": "UUID",
  "course_id": "MATH_10A_20260420",
  "timestamp": "2026-04-20T14:00:30Z",
  "sequence_number": 1,
  "image_path": "s3://bucket/course/frame_001.jpg",
  "image_size_kb": 512,
  "is_valid": true,
  "error_message": null
}
```

4.1.2 识别结果数据 (RecognitionResult)

```
{
  "frame_id": "UUID",
  "timestamp": "2026-04-20T14:00:30Z",
  "model_used": "qwen-v1-max",
  "model_version": "2026-04-01",
  "recognition_results": {
    "hand_raising": 2,
    "sleeping": 1,
    "phone_usage": 3,
    "note_taking": 20,
    "total_students": 26
  },
}
```

```
"confidence": 0.87,  
"processing_time_ms": 1250,  
"status": "success",  
"retry_count": 0  
}
```

4.1.3 统计数据 (Statistics)

```
{  
  "course_id": "MATH_10A_20260420",  
  "course_name": "高等数学",  
  "class_size": 26,  
  "start_time": "2026-04-20T14:00:00Z",  
  "end_time": "2026-04-20T14:45:00Z",  
  "duration_minutes": 45,  
  "total_samples": 90,  
  "valid_samples": 88,  
  "invalid_samples": 2,  
  "sampling_interval_sec": 30,  
  "statistics": {  
    "focus_rate": 76.92,  
    "distraction_rate": 15.38,  
    "activity_rate": 7.69,  
    "avg_hand_raising": 2.0,  
    "avg_sleeping": 0.4,  
    "avg_phone_usage": 0.8,  
    "avg_note_taking": 20.0  
  },  
  "trend_data": [  
    {"timestamp": "2026-04-20T14:00:30Z", "focus_rate": 73.08},  
    {"timestamp": "2026-04-20T14:01:00Z", "focus_rate": 76.92}  
  ],  
  "generated_at": "2026-04-20T14:50:00Z"  
}
```

4.1.4 报告数据 (Report)

```
{  
  "report_id": "UUID",  
  "course_id": "MATH_10A_20260420",  
  "generated_at": "2026-04-20T14:50:00Z",  
  "report_content": "文本内容...",  
  "current_stats_id": "stats-UUID",  
  "previous_stats_id": "stats-UUID-prev",  
  "status": "completed"  
}
```

4.2 数据存储方案

4.2.1 存储架构

云端存储架构：

- ├─ 结构化数据（使用关系数据库或NoSQL）
 - ├─ 采样帧元数据
 - ├─ 识别结果
 - ├─ 统计数据
 - └─ 报告内容
- ├─ 非结构化数据（对象存储）
 - └─ 采样图片（S3/OSS）
- └─ 缓存层（Redis）
 - └─ 实时处理状态、当前课程信息

4.2.2 数据库表设计

表1: courses (课程表)

```
CREATE TABLE courses (  
  course_id VARCHAR(50) PRIMARY KEY,  
  course_name VARCHAR(100),  
  class_size INT,  
  start_time TIMESTAMP,  
  end_time TIMESTAMP,  
  duration_minutes INT,  
  status VARCHAR(20),  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
);
```

表2: sampling_frames (采样帧表)

```
CREATE TABLE sampling_frames (  
  frame_id VARCHAR(50) PRIMARY KEY,  
  course_id VARCHAR(50) FOREIGN KEY,  
  sequence_number INT,  
  timestamp TIMESTAMP,  
  image_path VARCHAR(200),  
  image_size_kb INT,  
  is_valid BOOLEAN,  
  error_message TEXT,  
  created_at TIMESTAMP,  
  INDEX idx_course_time (course_id, timestamp)  
);
```

表3: recognition_results (识别结果表)

```
CREATE TABLE recognition_results (  
  result_id VARCHAR(50) PRIMARY KEY,  
  frame_id VARCHAR(50) FOREIGN KEY,  
  course_id VARCHAR(50) FOREIGN KEY,  
  timestamp TIMESTAMP,  
  model_used VARCHAR(50),  
  model_version VARCHAR(20),  
  hand_raising INT,  
  sleeping INT,  
  phone_usage INT,  
  note_taking INT,  
  total_students INT,  
  confidence FLOAT,  
  processing_time_ms INT,  
  status VARCHAR(20),  
  retry_count INT,  
  created_at TIMESTAMP,  
  INDEX idx_course_time (course_id, timestamp)  
);
```

表4: course_statistics (课程统计表)

```
CREATE TABLE course_statistics (  
  stats_id VARCHAR(50) PRIMARY KEY,  
  course_id VARCHAR(50) UNIQUE FOREIGN KEY,  
  class_size INT,  
  total_samples INT,  
  valid_samples INT,  
  invalid_samples INT,  
  focus_rate FLOAT,  
  distraction_rate FLOAT,  
  activity_rate FLOAT,  
  avg_hand_raising FLOAT,  
  avg_sleeping FLOAT,  
  avg_phone_usage FLOAT,  
  avg_note_taking FLOAT,  
  generated_at TIMESTAMP,  
  created_at TIMESTAMP,  
  UNIQUE KEY uk_course (course_id)  
);
```

表5: trend_data (趋势数据表)

```
CREATE TABLE trend_data (  
  trend_id VARCHAR(50) PRIMARY KEY,  
  course_id VARCHAR(50) FOREIGN KEY,
```

```
timestamp TIMESTAMP,
focus_rate FLOAT,
distraction_rate FLOAT,
activity_rate FLOAT,
created_at TIMESTAMP,
INDEX idx_course_time (course_id, timestamp)
);
```

表6: reports (报告表)

```
CREATE TABLE reports (
  report_id VARCHAR(50) PRIMARY KEY,
  course_id VARCHAR(50) FOREIGN KEY,
  current_stats_id VARCHAR(50) FOREIGN KEY,
  previous_stats_id VARCHAR(50) FOREIGN KEY,
  report_content LONGTEXT,
  generated_at TIMESTAMP,
  created_at TIMESTAMP,
  INDEX idx_course (course_id)
);
```

4.3 数据保留策略

数据类型	保留期限	备注
采样图片	7天	存储在对象存储中，超期自动删除
采样帧元数据	1个月	用于重新处理或问题追踪
识别结果	1个月	完整的行为数据
统计数据	长期保存	用于历史对比分析
报告	长期保存	用于查询和审计

API设计

5.1 OpenClaw集成接口

5.1.1 启动课程处理任务

```
POST /api/v1/course/start
Content-Type: application/json

请求体:
{
  "course_id": "MATH_10A_20260420",
  "course_name": "高等数学",
  "rtsp_url": "rtsp://192.168.0.136:8554/1027116728",
```

```
"class_size": 26,  
"teacher_id": "T001",  
"classroom_id": "101"  
}
```

响应:

```
{  
  "task_id": "TASK_UUID",  
  "status": "started",  
  "message": "课程处理已启动",  
  "timestamp": "2026-04-20T14:00:00Z"  
}
```

5.1.2 停止课程处理任务

POST /api/v1/course/stop
Content-Type: application/json

请求体:

```
{  
  "course_id": "MATH_10A_20260420",  
  "generate_report": true  
}
```

响应:

```
{  
  "course_id": "MATH_10A_20260420",  
  "status": "stopped",  
  "total_frames_processed": 90,  
  "valid_frames": 88,  
  "invalid_frames": 2,  
  "report_id": "REPORT_UUID",  
  "timestamp": "2026-04-20T14:50:00Z"  
}
```

5.1.3 获取课程实时状态

GET /api/v1/course/{course_id}/status

响应:

```
{  
  "course_id": "MATH_10A_20260420",  
  "status": "processing",  
  "elapsed_time_seconds": 1800,  
  "frames_processed": 45,  
  "frames_failed": 1,  
  "current_fps": 0.025,  
  "connection_status": "connected",  
}
```

```
"last_sample_time": "2026-04-20T14:30:00Z"
}
```

5.1.4 获取课程统计数据

GET /api/v1/course/{course_id}/statistics

响应:

```
{
  "course_id": "MATH_10A_20260420",
  "course_name": "高等数学",
  "class_size": 26,
  "total_samples": 90,
  "valid_samples": 88,
  "invalid_samples": 2,
  "statistics": {
    "focus_rate": 76.92,
    "distraction_rate": 15.38,
    "activity_rate": 7.69,
    "avg_hand_raising": 2.0,
    "avg_sleeping": 0.4,
    "avg_phone_usage": 0.8,
    "avg_note_taking": 20.0
  },
  "generated_at": "2026-04-20T14:50:00Z"
}
```

5.1.5 获取课程报告

GET /api/v1/course/{course_id}/report

响应:

```
{
  "report_id": "REPORT_UUID",
  "course_id": "MATH_10A_20260420",
  "generated_at": "2026-04-20T14:50:00Z",
  "report_content": "文本内容...",
  "with_comparison": true,
  "previous_course_id": "MATH_10A_20260419"
}
```

5.1.6 获取历史课程对比

GET /api/v1/course/{course_id}/comparison?previous_course_id={previous_course_id}

响应:


```
{
  "current_course": {
    "course_id": "MATH_10A_20260420",
    "focus_rate": 76.92,
    "distraction_rate": 15.38,
    "activity_rate": 7.69
  },
  "previous_course": {
    "course_id": "MATH_10A_20260419",
    "focus_rate": 73.77,
    "distraction_rate": 17.54,
    "activity_rate": 7.69
  },
  "changes": {
    "focus_rate": "+3.15%",
    "distraction_rate": "-2.16%",
    "activity_rate": "0%"
  }
}
```

5.2 视觉大模型调用接口

5.2.1 通用调用模板

```
# 基于OpenClaw的任务编排
task_config = {
  "task_name": "recognize_classroom_behavior",
  "model": "vision_model_api",
  "input": {
    "image": "base64_encoded_image",
    "prompt": "请分析当前课堂画面中学生行为..."
  },
  "timeout": 30,
  "retry_policy": {
    "max_retries": 3,
    "backoff": "exponential",
    "initial_delay": 2
  }
}
```

大模型选择

6.1 推荐模型对比

基于系统需求（准确率优先、API调用方式、支持视觉识别），推荐3个核心模型：

6.1.1 方案A: Qwen VL系列（推荐）

指标	评分	说明
模型	★★★★★	Qwen2.5-VL 或 Qwen3-VL
准确率	★★★★★	业界领先的视觉理解能力
响应速度	★★★★	通常在1-3秒内返回
API完整度	★★★★★	完整的API文档和SDKs
成本	★★★	中等成本 (¥0.006-0.012/k tokens)
集成复杂度	★★★★	中等，有官方Python SDK

优势:

- 中文理解能力最强
- 指令遵循能力强
- 对教室场景优化较好
- 国内厂商，技术支持好

劣势:

- 相比GPT-4V稍弱

推荐理由: 最适合国内教学场景，中文理解最佳

6.1.2 方案B: GPT-4V

指标	评分	说明
模型	★★★★★	GPT-4V/GPT-4 Turbo Vision
准确率	★★★★★	业界最强
响应速度	★★★	较慢，2-5秒
API完整度	★★★★★	完整的API
成本	★	最贵 (\$0.01-0.03/image)
集成复杂度	★★★★	中等

优势:

- 准确率最高
- 通用性最强
- 文档完善

劣势:

- 成本最高
- 响应较慢
- 调用次数可能被限制

推荐理由: 如果需要业界最强准确率，且成本不是问题

6.1.3 方案C: InternVL

指标	评分	说明
模型	★★★★	InternVL-Chat-V2系列
准确率	★★★★	接近GPT-4V
响应速度	★★★★	1-2秒
API完整度	★★★	基础完整
成本	★★★★	较低（按需定价）
集成复杂度	★★★	中等

优势:

- 综合性价比高
- 速度快
- 学术背景强

劣势:

- 中文理解略弱于Qwen
- 文档不如GPT-4V完善

6.2 推荐选型方案

优先级方案（从上到下）：

第1选：Qwen3-VL（或Qwen2.5-VL）

- └ 理由：中文理解+响应速度+成本平衡最优
- └ 适合：国内教学场景
- └ 成本：中等

第2选：GPT-4V

- └ 理由：最高准确率
- └ 适合：对准确率有极端要求
- └ 成本：最高

第3选：InternVL

- └ 理由：性价比替代方案
- └ 适合：成本敏感但要求较高准确率
- └ 成本：低

建议: 使用 **Qwen3-VL** 作为主模型，留出 **GPT-4V** 作为备选方案

部署方案

7.1 部署架构



7.2 部署清单

组件	部署方式	规格	备注
OpenClaw	Docker容器	2C 4G	单摄像头可用此规格
API服务	Docker容器	2C 4G	支持水平扩展
MySQL	云数据库RDS	1C 2G	可根据数据量调整
Redis	云缓存服务	1G	缓存实时状态
对象存储	S3/OSS	-	存储采样图片
日志采集	ELK/Datadog	-	监控和调试

错误处理与恢复

8.1 分层错误处理

8.1.1 视频流连接层

场景1：初次连接失败

- 重试间隔：2秒
- 最大重试：5次
- 失败处理：报错，等待人工干预

场景2：连接中途断开

- └ 自动重连
- └ 重试策略：指数退避（2s, 4s, 8s, 16s, 32s）
- └ 最大重试：5次
- └ 断开时长统计：记录到日志

场景3：部分帧丢失

- └ 继续处理
- └ 统计无效帧数
- └ 最后报告中标注

8.1.2 视觉模型调用层

场景1：API调用超时

- └ 重试次数：最多3次
- └ 重试延迟：2s, 5s, 10s
- └ 最终失败：记录错误，跳过该帧

场景2：API返回错误码

- └ 4xx错误：参数问题，记录后跳过该帧
- └ 5xx错误：服务故障，重试
- └ 限流(429)：等待后重试

场景3：返回结果异常

- └ JSON解析失败：记录原始响应，跳过该帧
- └ 字段缺失：尝试补填，或跳过
- └ 数值异常：验证后接受或丢弃

8.1.3 数据处理层

场景1：统计计算异常

- └ 无有效数据：报告为"数据不足"
- └ 数据异常：使用中位数替代异常值
- └ 计算溢出：记录警告，使用安全值

场景2：数据库写入失败

- └ 连接失败：重试，最多3次
- └ 超时：本地缓存后异步重试
- └ 约束冲突：记录警告并跳过

8.2 错误恢复策略

级别1：自动重试（透明恢复）

- └ 不需要人工干预

级别2：降级处理（部分可用）

- └ 继续处理，记录问题帧
- └ 报告中标注数据质量

级别3：报错停止（需要干预）

- └ 记录完整错误信息
- └ 通过日志系统告警
- └ 等待人工检查和重启

8.3 错误日志示例

```
{
  "timestamp": "2026-04-20T14:30:45Z",
  "course_id": "MATH_10A_20260420",
  "error_type": "vision_model_timeout",
  "error_level": "warning",
  "frame_id": "frame_045",
  "details": {
    "model": "qwen-vl-max",
    "request_timeout": "30s",
    "retry_count": 3,
    "action_taken": "skipped_frame"
  },
  "context": {
    "elapsed_time": 1350,
    "total_frames_processed": 45,
    "valid_frames": 44,
    "failed_frames": 1
  }
}
```

日志与调试

9.1 日志设计

9.1.1 日志分级

级别	说明	示例
DEBUG	详细调试信息	帧采样、API调用详情
INFO	一般信息	课程启动、帧处理统计
WARN	警告信息	部分帧失败、连接瞬断
ERROR	错误信息	连接断开、模型失败
FATAL	致命错误	无法恢复的系统故障

9.1.2 日志字段

```
{
  "timestamp": "2026-04-20T14:30:45Z",
  "level": "INFO",
  "component": "opencv_sampling",
  "course_id": "MATH_10A_20260420",
  "event": "frame_sampled",
  "details": {
    "frame_id": "frame_045",
    "sequence_number": 45,
    "image_size_kb": 512,
    "processing_time_ms": 50
  },
  "context": {
    "total_frames": 90,
    "success_frames": 44,
    "failed_frames": 1
  }
}
```

9.1.3 关键日志事件

课程生命周期：

- └─ [INFO] 课程启动
 - └─ course_id, rtsp_url, class_size
- └─ [INFO] RTSP连接成功
 - └─ connection_details
- └─ [INFO] 开始采样循环
 - └─ sampling_interval
- └─ [INFO] 帧采样完成（每30秒）
 - └─ frame_info, size, timestamp
- └─ [INFO] 行为识别完成
 - └─ recognition_results, confidence
- └─ [WARN/ERROR] 异常事件
 - └─ error_type, retry_info
- └─ [INFO] 课程结束，报告生成
 - └─ stats_summary, report_id

9.2 调试接口

9.2.1 实时状态查询

GET /api/v1/debug/course/{course_id}

响应内容：

```
{
  "course_status": {
    "is_running": true,
```

```
    "uptime_seconds": 1800,
    "connection_status": "connected",
    "last_heartbeat": "2026-04-20T14:30:45Z"
  },
  "processing_stats": {
    "frames_sampled": 45,
    "frames_processed": 44,
    "frames_failed": 1,
    "avg_processing_time_ms": 1250
  },
  "recent_errors": [
    {
      "timestamp": "2026-04-20T14:29:00Z",
      "error_type": "vision_model_timeout",
      "frame_id": "frame_043"
    }
  ]
}
```

9.2.2 日志查询接口

GET /api/v1/debug/logs?course_id={id}&level=ERROR&limit=100

响应：返回最近100条ERROR级别日志

9.2.3 手动重试接口

POST /api/v1/debug/course/{course_id}/retry_frame

请求：

```
{
  "frame_id": "frame_043"
}
```

响应：

```
{
  "frame_id": "frame_043",
  "status": "retry_queued",
  "message": "帧已加入重试队列"
}
```

实施计划

10.1 开发阶段

第1阶段: 基础架构搭建 (1周)

- ☐ 搭建OpenClaw开发环境
- ☐ 配置云数据库和对象存储
- ☐ 实现Miloco视频流对接
- ☐ 建立基本日志系统

第2阶段: 核心功能开发 (2周)

- ☐ 实现视频采样模块（RTSP拉流、30s采样）
- ☐ 实现重连机制
- ☐ 集成视觉大模型API
- ☐ 实现统计计算模块

第3阶段: 报告生成与历史对比 (1周)

- ☐ 实现报告生成模块
- ☐ 实现历史对比功能
- ☐ 优化报告文案

第4阶段: API和管理接口 (1周)

- ☐ 开发REST API
- ☐ 实现调试接口
- ☐ 编写API文档

第5阶段: 测试与优化 (2周)

- ☐ 单元测试
- ☐ 集成测试
- ☐ 性能测试
- ☐ 错误场景测试

第6阶段: 部署和文档 (1周)

- ☐ 容器化部署
- ☐ 编写运维文档
- ☐ 用户操作手册

10.2 关键里程碑

里程碑	日期	交付物
基础架构完成	T+7	开发环境、数据库、日志系统
原型系统可用	T+21	能完整处理一节课
API接口完整	T+28	所有REST接口可用
MVP版本上线	T+42	可用于小规模试验

10.3 技术栈推荐

模块	技术栈	备注
编排引擎	OpenClaw (Python/C++)	核心框架
Web框架	FastAPI 或 Flask	API服务
数据库	PostgreSQL 或 MySQL	关系数据型数据
缓存	Redis	实时状态缓存
存储	AWS S3 / 阿里OSS	对象存储
日志	ELK Stack 或 Datadog	日志收集与分析
容器	Docker	应用容器化
版本控制	Git	代码管理

附录

A. 术语表

术语	说明
RTSP	实时流传输协议 (Real Time Streaming Protocol)
采样间隔	相邻两次截图的时间间隔
专注率	记笔记学生数 / 总学生数
分心率	(睡觉+玩手机) / 总学生数
活跃度	举手学生数 / 总学生数
API	应用编程接口
云端存储	基于云服务的数据存储 (S3/OSS)

B. FAQ

- Q: 如果视频流断开会怎样?** A: 系统会自动尝试重连, 采用指数退避策略最多重试5次。期间采样暂停, 断开期间的数据记为无效。
- Q: 识别准确率多高?** A: 取决于选用的视觉模型。推荐的Qwen3-VL在教室场景下准确率约85-90%。
- Q: 支持离线部署吗?** A: 暂不支持。当前设计基于云端API调用, 需要互联网连接。
- Q: 数据会保存多久?** A: 采样图片保存7天, 分析数据保存1个月, 统计结果长期保存。
- Q: 可以识别特定学生吗?** A: 当前不支持学生ID识别, 仅统计行为总数。后续可扩展支持。

文档完成日期: 2026年4月20日
版本: 1.0
状态: 待审批